Name: Chinta Lakshmi Nivas Jaswanth

Module 2

1.E-commerce Platform Search Function

Understanding Asymptotic Notation

What is Big O Notation?

Big O notation represents how the execution time of an algorithm grows as the input size increases.

Examples:

- O(1) → Constant Time
- O(log n) → Logarithmic Time
- O(n) → Linear Time
- O(n log n) → Linearithmic Time
- O(n^2) → Quadratic Time

Big O helps developers compare algorithms and choose the most efficient one for large datasets.

Search Operation Cases

Best Case

The element is found immediately.

Example:

- Searching first element in an array.

Linear Search: O(1)

Binary Search: O(1)

Average Case

The element is found somewhere in the middle.

Linear Search: O(n)

Binary Search: O(log n)

Worst Case

The element is at the end or not present.

Linear Search: O(n)

Binary Search: O(log n)

Product class:

```
class Product {
    int productId;
    String productName;
    String category;
    Product(int productId, String productName, String category) {
        this.productId = productId;
        this.productName = productName;
        this.category = category;
    }
    public void display() {
        System.out.println("ID: " + productId +
```

```
                    ", Name: " + productName +
                    ", Category: " + category);
    }
}
```

Linear Search implementation:

```
class LinearSearch {
    public static Product search(Product[] products, int targetId) {
        for (Product product : products) {
            if (product.productId == targetId) {
                return product;
            }
        }
        return null;
    }
}
```

Binary Search implementation:

```
class BinarySearch {
    public static Product search(Product[] products, int targetId) {
        int low = 0;
        int high = products.length - 1;

        while (low <= high) {
```

```
            int mid = (low + high) / 2;
            if (products[mid].productId == targetId) {
                return products[mid];
            }
            if (products[mid].productId < targetId) {
                low = mid + 1;
            } else {
                high = mid - 1;
            }
        }
        return null;
    }
}
```

Main Program:

```
public class EcommerceSearch {
    public static void main(String[] args) {
        Product[] products = {
            new Product(101, "Laptop", "Electronics"),
            new Product(102, "Mobile", "Electronics"),
            new Product(103, "Shoes", "Fashion"),
            new Product(104, "Watch", "Accessories"),
            new Product(105, "Bag", "Fashion")
        };
```

```
        int searchId = 104;
        System.out.println("Linear Search:");
        Product result1 = LinearSearch.search(products, searchId);
        if (result1 != null) {
            result1.display();
        } else {
            System.out.println("Product not found");
        }
        System.out.println("\nBinary Search:");
        Product result2 = BinarySearch.search(products, searchId);
        if (result2 != null) {
            result2.display();
        } else {
            System.out.println("Product not found");
        }
    }
}
```

Time Complexity Comparison

Algorithm	Best Case	Average Case	Worst Case
Linear Search	O(1)	O(n)	O(n)
Binary Search	O(1)	O(log n)	O(log n)

Which Algorithm is Better?

Linear Search

Advantages:

- Simple to implement.
- Works on unsorted data.

Disadvantages:

- Slow for large product catalogs.

Binary Search

Advantages:

- Much faster for large datasets.
- Efficient for e-commerce websites with thousands or millions of products.

Disadvantages:

- Requires data to be sorted.

Conclusion

For an e-commerce platform, **Binary Search is more suitable** because product databases are typically large. Its **O(log n)** time complexity makes searches significantly faster than Linear Search's **O(n)** complexity. However, products must be maintained in sorted order for Binary Search to work correctly.

2) Financial Forecasting Using Recursion

Understanding Recursive Algorithms

What is Recursion?

Recursion is a programming technique where a method calls itself to solve a problem by breaking it into smaller subproblems.

Every recursive function has:

1. Base Case – stops recursion.
2. Recursive Case – calls itself with a smaller problem.

Example

Factorial:

$$5! = 5 \times 4!$$
$$4! = 4 \times 3!$$

and so on until:

$$1! = 1$$

Recursion simplifies problems that have a repetitive structure.

---

2. Setup

Suppose:

- Present Value = ₹10,000
- Annual Growth Rate = 10%
- Years = 5

Future value formula:

$$FV = PV \times (1 + r)^n$$

Where:

- FV = Future Value
- PV = Present Value
- r = Growth Rate
- n = Number of Years

Using recursion:

$$FV(n) = FV(n-1) \times (1+r)$$

Base Case:

$$FV(0) = PV$$

Java Program:

```
public class FinancialForecasting {
    public static double predictFutureValue(double currentValue,
                                            double growthRate,
                                            int years) {
        if (years == 0) {
            return currentValue;
        }
        return predictFutureValue(
                currentValue * (1 + growthRate),
                growthRate,
                years - 1);
    }
```

```
    public static void main(String[] args) {
        double presentValue = 10000;
        double growthRate = 0.10;
        int years = 5;
        double futureValue = predictFutureValue(
                presentValue,
                growthRate,
                years);
        System.out.println("Future Value after "
                + years + " years = ₹"
                + futureValue);
    }
}
```

Analysis

Time Complexity

For every year, one recursive call is made.

$$T(n) = T(n-1) + 1$$

Therefore:

$$T(n) = O(n)$$

Time Complexity = O(n)

Space Complexity = O(n)

because the recursive call stack stores n function calls.

Optimization : Memoization

Store already-computed results in an array or HashMap.

This avoids recalculating the same values multiple times when recursive subproblems overlap.

Comparison

Method	*Time Complexity*	*Space Complexity*
Recursive	*O(n)*	*O(n)*
Memoized Recursive	*O(n)*	*O(n)*
Mathematical Formula	*O(1)*	*O(1)*

Conclusion

- *Recursion solves the forecasting problem by repeatedly applying the growth rate until the required number of years is reached.*
- *The recursive solution has O(n) time complexity and O(n) space complexity.*
- *For real-world financial forecasting systems, using the compound growth formula (Math.pow) is usually the most efficient approach because it avoids deep recursion and executes much faster.*